From Linear Classifiers to Neural Networks

Chapter 2 · Neurons, activations, hidden representations, and the forward pass

Dr. A. Belcaid

Table of contents

1 From a linear score to a neuron **2**
1.1 The computation performed by one neuron . . . 4
1.2 Why nonlinearities are necessary . . . 5

2 Activation functions **5**
2.1 Sigmoid . . . 6
2.2 Hyperbolic tangent . . . 6
2.3 Rectified linear unit . . . 7
2.4 A controlled activation experiment . . . 9

3 Building a two-layer network **9**
3.1 Shape bookkeeping . . . 10
3.2 Counting parameters . . . 10

4 Hidden representations **11**
4.1 Filters and feature maps are different objects . . . 12
4.2 A representation is a pattern of activations . . . 12
4.3 Width and depth play different roles . . . 12

5 Implementing the forward pass in NumPy **14**
5.1 Defensive shape checks . . . 15
5.2 Sanity checks before training . . . 15

6 What the forward pass does not yet solve **16**

7 Summary **16**

8 Reading and practice **17**

References **17**

PART II · NEURAL NETWORKS FROM SCRATCH

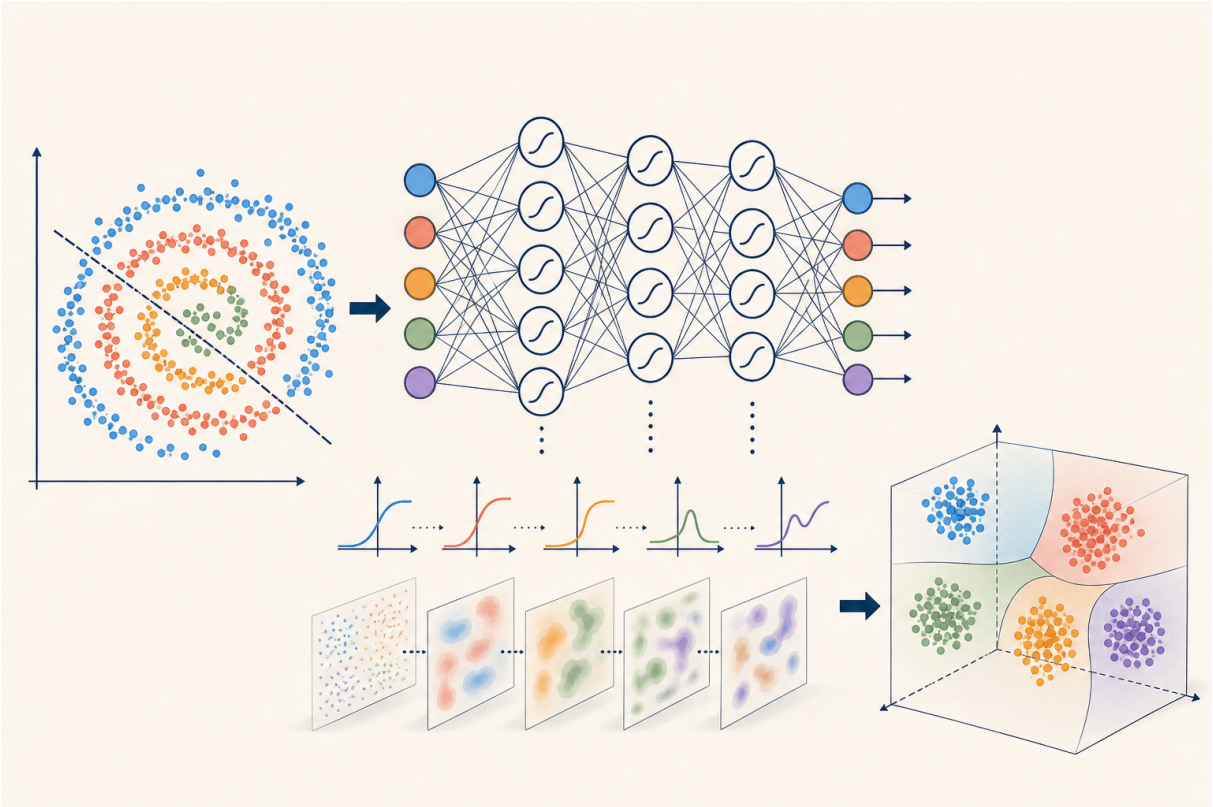


Figure 1: Nonlinearly entangled data passes through neurons and successive feature transformations before becoming separable in a learned hidden space. Original course illustration generated for this chapter.

The answer is not “add many matrices.” Several affine maps in succession still collapse into one affine map. A neural network gains expressive power by placing a **nonlinear activation** between learned affine transformations.

By the end of this chapter, you should be able to

1. Describe a neuron as an affine score followed by an activation.
2. Explain geometrically why a single linear classifier fails on nonlinear data.
3. Compute sigmoid, tanh, and ReLU outputs and derivatives.
4. Write the forward pass of a two-layer multilayer perceptron using matrices.
5. Check every parameter and intermediate tensor shape.
6. Explain how hidden activations form a learned representation.
7. Implement a self-contained NumPy forward pass and prediction method.

1 From a linear score to a neuron

Chapter 1 introduced the multiclass linear classifier

$$S = XW + b.$$

For a batch of N examples with D input features and K classes,

$$X \in \mathbb{R}^{N \times D}, \qquad W \in \mathbb{R}^{D \times K}, \qquad b \in \mathbb{R}^{K}, \qquad S \in \mathbb{R}^{N \times K}.$$

Each column of W defines one class score. This model is efficient, but each pairwise decision boundary is a hyperplane. No amount of training can make one hyperplane wrap around a curved or disconnected class.

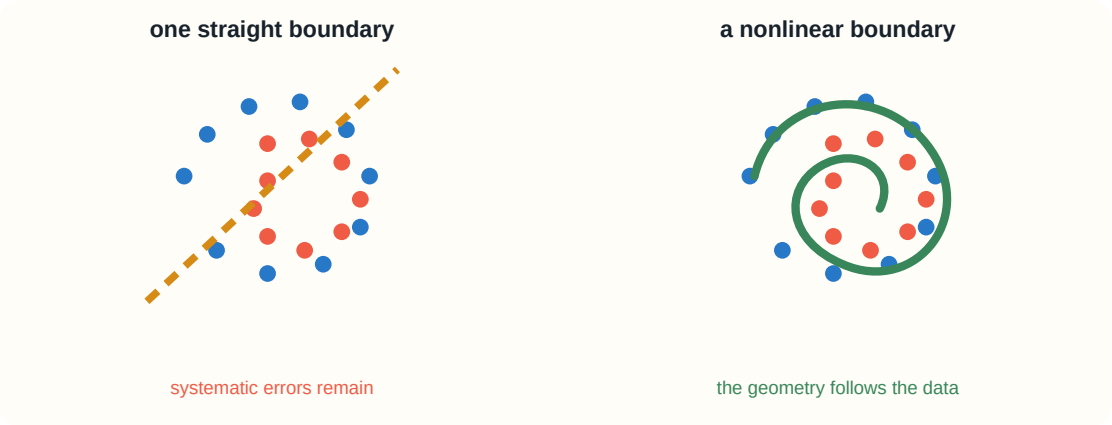


Figure 2: A linear boundary cannot separate the intertwined spiral classes, while a small neural network can construct a nonlinear partition. The drawing is based on the Stanford CS231n spiral case study (Li et al., n.d.).

The TensorFlow Playground makes this limitation visible. Select the concentric-circle dataset, first train a linear model, and then add a hidden layer. The hidden layer can transform the input before the final linear decision.

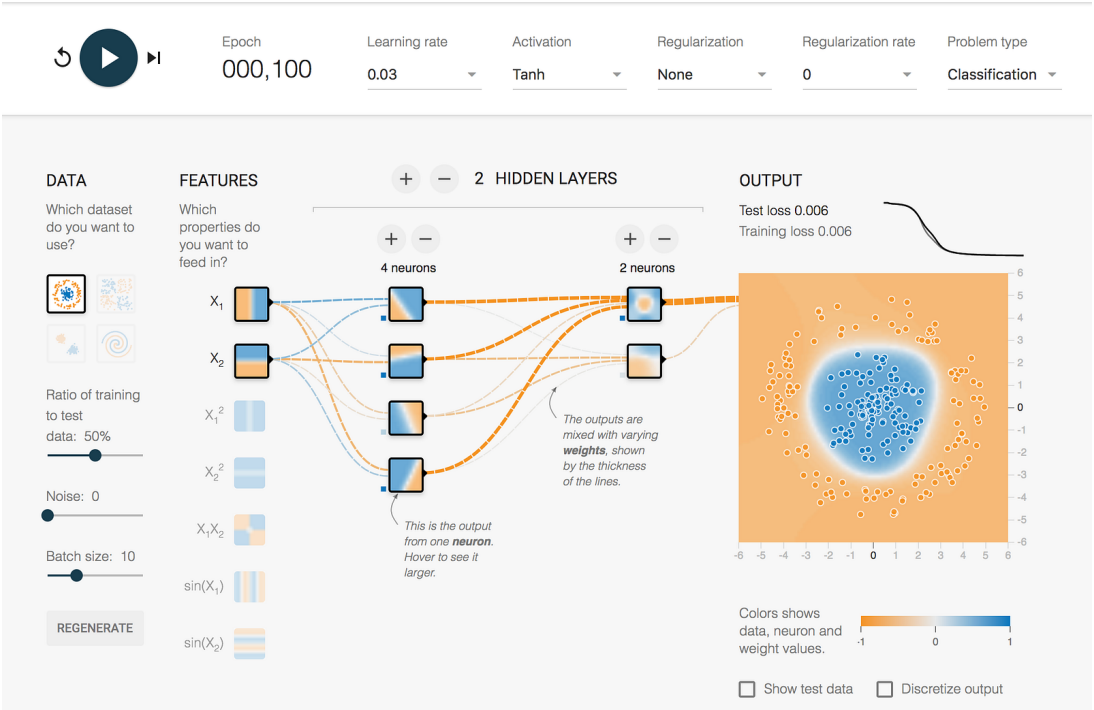


Figure 3: TensorFlow Playground fitting concentric classes with a neural network. The visualization exposes the learned output of each hidden unit.

1.1 The computation performed by one neuron

A neuron receives an input vector $x \in \mathbb{R}^D$, computes a weighted sum plus a bias,

$$z = w^\top x + b,$$

and applies an activation function ϕ:

$$h = \phi(z).$$

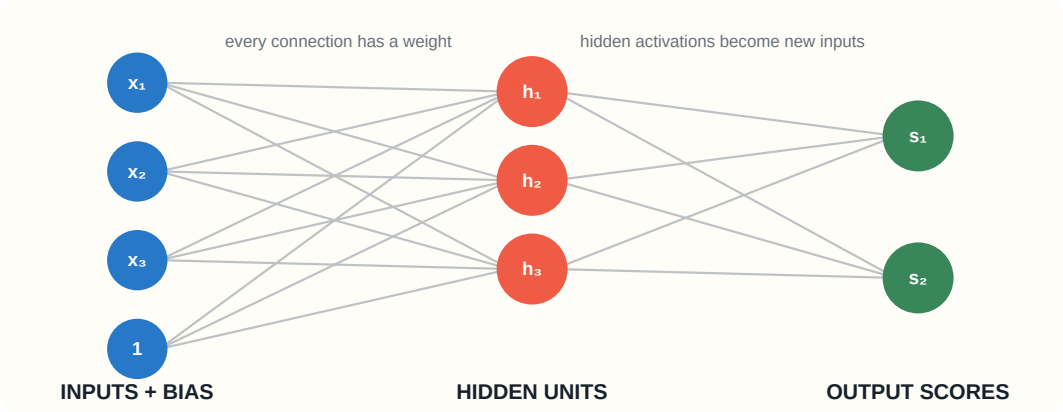


Figure 4: Two complementary views of a neuron: incoming features are weighted and summed, then an activation transforms the pre-activation z into an output h. Course diagram.

The weights control orientation, the bias controls translation, and the activation controls the form of the response. Before the activation, the equation

$$w^\top x + b = 0$$

defines a hyperplane. The signed value $z = w^\top x + b$ indicates on which side of that hyperplane the input lies and how strongly it matches the neuron's learned direction.

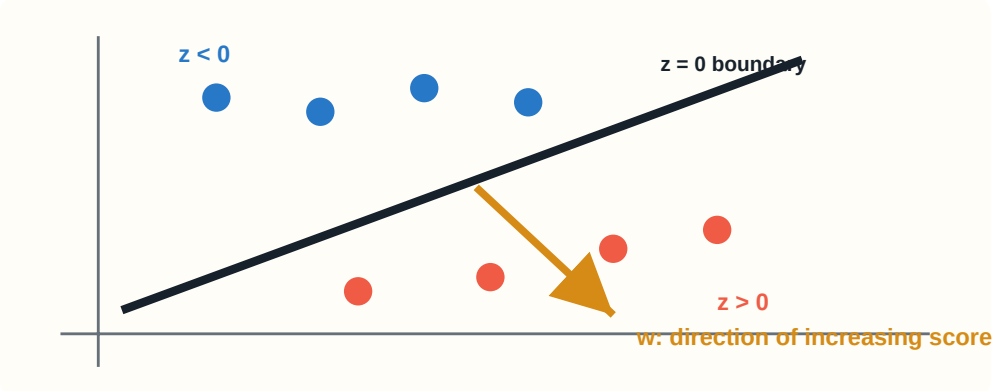


Figure 5: A neuron first creates a linear boundary and then converts signed distance-like scores into responses. Course schematic; the biological-neuron analogy commonly used in neural-network introductions is also illustrated by Stanford CS231n (Li et al., n.d.).

The neuron picture is useful, but the biological analogy should not be taken literally. Artificial neurons are differentiable computational units inspired only loosely by biological cells. Their practical importance comes from composition, matrix computation, and optimization—not biological fidelity (Goodfellow et al. 2016).

1.2 Why nonlinearities are necessary

Suppose we stack two affine transformations without an activation:

$$f(x) = (xW_1 + b_1)W_2 + b_2.$$

Expanding gives

$$f(x) = x(W_1W_2) + (b_1W_2 + b_2).$$

Define $W' = W_1W_2$ and $b' = b_1W_2 + b_2$. Then

$$f(x) = xW' + b',$$

which is still one affine transformation. Ten affine layers without nonlinearities have the same structural limitation.

Depth alone does not create nonlinear behavior. The activation prevents successive layers from collapsing into a single affine map.

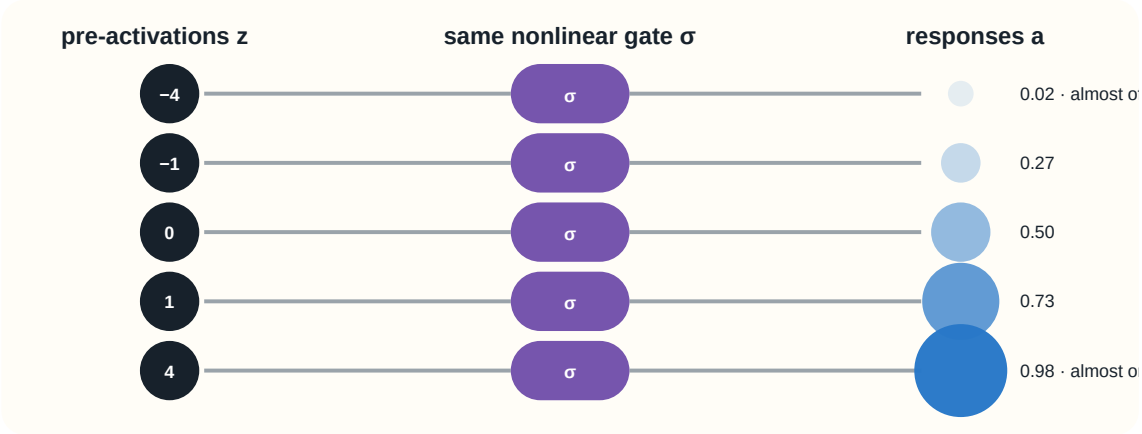


Figure 6: Several neurons apply the same activation to different pre-activations. Some units respond strongly and others are suppressed, creating a new feature vector rather than another linear copy of the input.

2 Activation functions

An activation function should be understood through four views: its formula, its output curve, its derivative, and the optimization behavior produced by that derivative.

2.1 Sigmoid

The logistic sigmoid is

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

Its derivative can be expressed using its own output:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

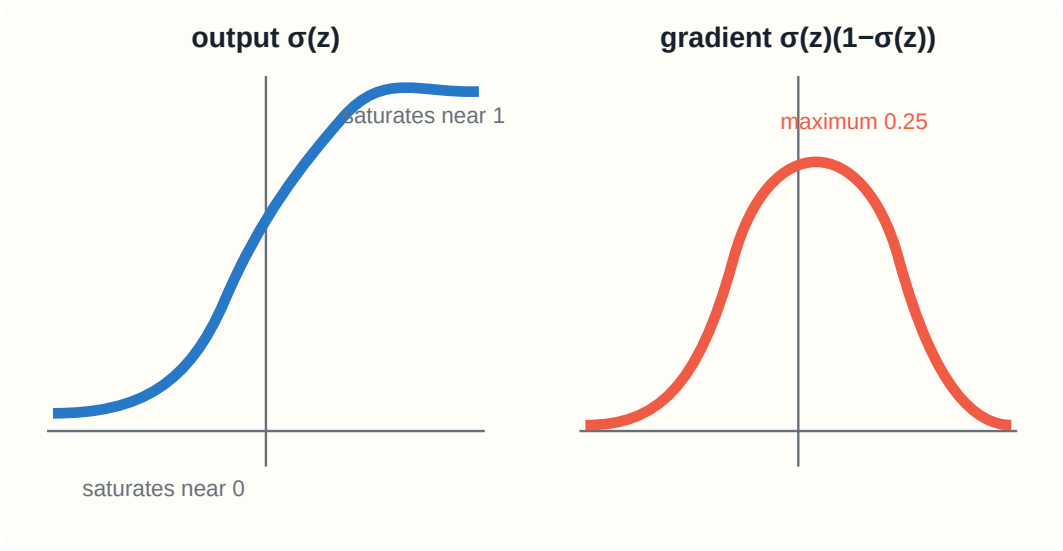


Figure 7: Sigmoid output and derivative. The derivative is at most $1/4$ and approaches zero for inputs far from the origin.

Sigmoid maps every real number into $(0, 1)$, so it is natural for the output of a binary probabilistic classifier. As a hidden activation, however, it has two important weaknesses:

- it is not zero-centered;
- it saturates for large positive or negative inputs, where its derivative is nearly zero.

When many saturated units are composed, gradients passed backward through their derivatives can shrink severely. We will formalize this phenomenon when deriving backpropagation.

2.2 Hyperbolic tangent

The hyperbolic tangent is

$$\tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}},$$

with derivative

$$\frac{d}{dz}\tanh(z) = 1 - \tanh^2(z).$$

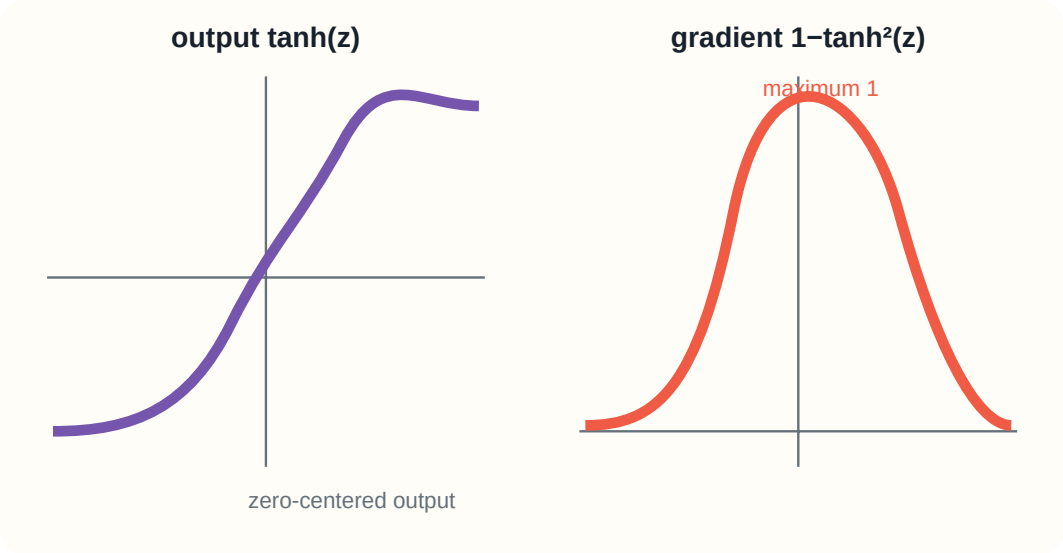


Figure 8: Tanh output and derivative. Tanh is zero-centered but still saturates at both extremes.

Tanh maps to $(-1, 1)$ and is zero-centered, which often makes its hidden activations easier to work with than sigmoid outputs. It nevertheless saturates, and its derivative also approaches zero when $|z|$ is large.

2.3 Rectified linear unit

The rectified linear unit, or ReLU, is

$$\mathrm{ReLU}(z) = \max(0, z).$$

Away from zero, its derivative is

$$\mathrm{ReLU}'(z) = \begin{cases} 0, & z < 0, \\ 1, & z > 0. \end{cases}$$

At $z = 0$ the mathematical derivative is undefined; implementations choose a conventional subgradient, commonly zero.

ReLU is computationally simple, does not saturate on its positive side, and often produces sparse activations. A unit can nevertheless become inactive for every training example if updates leave all its pre-activations negative—the “dying ReLU” problem. Variants such as leaky ReLU keep a small negative slope.

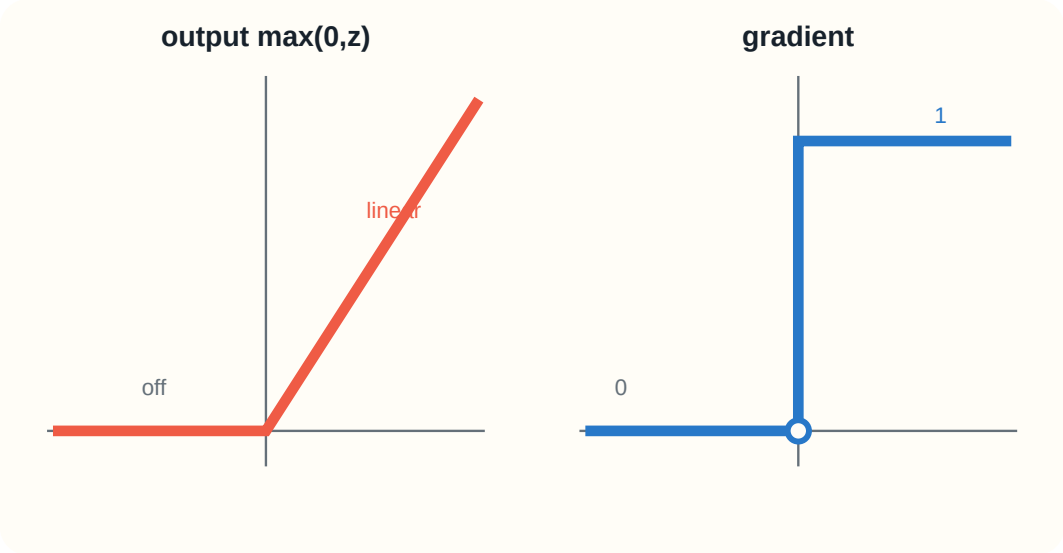


Figure 9: ReLU output and derivative. Positive values pass unchanged, while negative values and their local gradients are suppressed.

Activation	Range	Largest derivative	Main strength	Main weakness
Sigmoid	$(0, 1)$	0.25	Probability-like output	Saturation; not zero-centered
Tanh	$(-1, 1)$	1	Zero-centered	Saturation
ReLU	$[0, \infty)$	1	Simple; positive side does not saturate	Zero gradient on negative side

Practice before reading the solution

Apply ReLU elementwise to

$$z = [-2.0,\ -0.1,\ 0,\ 1.4,\ 3.0].$$

Which entries can pass a nonzero local gradient? Pause for one minute.

Reveal the solution

$$\mathrm{ReLU}(z) = [0, 0, 0, 1.4, 3.0].$$

The entries at 1.4 and 3.0 have local derivative one. Negative entries have derivative zero. At exactly zero, our implementation will use derivative zero.

2.4 A controlled activation experiment

In TensorFlow Playground, keep the dataset, train/validation split, architecture, learning rate, and number of training steps fixed. Change only the activation among sigmoid, tanh, and ReLU. Then inspect:

1. how rapidly the training loss falls;
2. the boundary each model learns;
3. which hidden units become nearly constant;
4. whether the outcome changes when training is restarted.

This is not a benchmark. It is a controlled experiment for connecting the shapes in Figures Figure 7–Figure 9 to learning behavior. One run is not sufficient evidence because initialization and stochastic sampling can change the result.

3 Building a two-layer network

A **layer** collects many neurons and evaluates them in parallel. Let the input dimension be D, the hidden width be H, and the number of output classes be K.

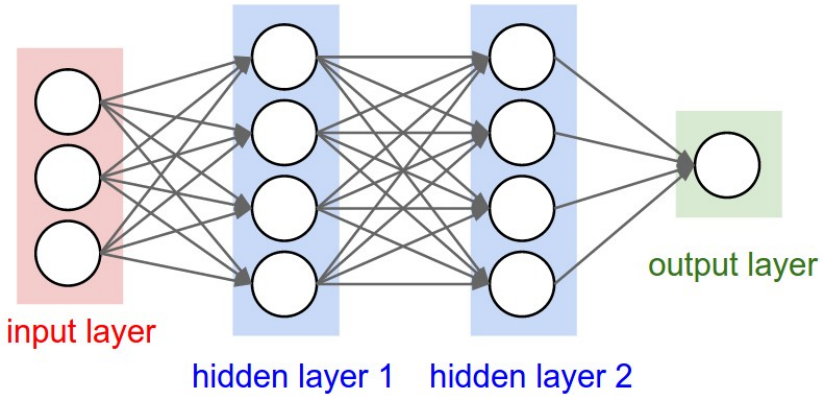


Figure 10: A fully connected neural network with one hidden layer. Every input connects to every hidden unit, and every hidden unit connects to every output. Stanford CS231n (Li et al., n.d.).

For a batch $X \in \mathbb{R}^{N \times D}$, define the forward pass as

$$\begin{aligned} Z_1 &= XW_1 + b_1, \\ H_1 &= \mathrm{ReLU}(Z_1), \\ S &= H_1 W_2 + b_2. \end{aligned}$$

Here Z_1 contains the **pre-activations**, H_1 contains the hidden activations, and S contains the class scores. We do not apply ReLU to S: the loss needs unrestricted class scores.

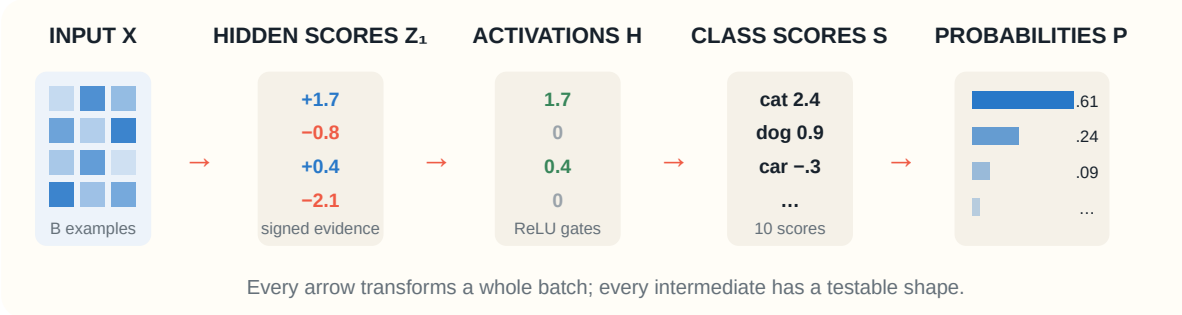


Figure 11: The two-layer forward pass shown as explicit computational stages, from a batch of inputs to class scores.

3.1 Shape bookkeeping

The shapes reveal whether each matrix product is valid:

Object	Shape	Meaning
X	$N \times D$	batch of N examples
W_1	$D \times H$	input-to-hidden weights
b_1	H	one bias per hidden unit
Z_1, H_1	$N \times H$	one hidden representation per example
W_2	$H \times K$	hidden-to-output weights
b_2	K	one bias per class
S	$N \times K$	one class-score vector per example

Bias vectors are broadcast across the N rows. Broadcasting does not create a separate learned bias for each example; every example uses the same b_1 and b_2.

Shape exercise

Let $N = 64$, $D = 3072$, $H = 100$, and $K = 10$. Determine the shapes of all parameters and intermediates before revealing the answer.

Reveal the solution

$$\begin{aligned} X &: (64, 3072), \quad W_1 : (3072, 100), \quad b_1 : (100), \\ Z_1, H_1 &: (64, 100), \quad W_2 : (100, 10), \quad b_2 : (10), \\ S &: (64, 10). \end{aligned}$$

3.2 Counting parameters

The number of learned scalar parameters is

$$P = (DH + H) + (HK + K).$$

For $D = 3072$, $H = 100$, and $K = 10$,

$$P = 3072(100) + 100 + 100(10) + 10 = 308{,}310.$$

If the hidden width doubles to $H = 200$, the hidden activations double in width and both weight matrices grow:

$$P = 3072(200) + 200 + 200(10) + 10 = 616{,}610.$$

The count is almost doubled because the dominant terms DH and HK are linear in H. Greater width creates more features and capacity, but also increases computation, memory use, and the opportunity to overfit.

4 Hidden representations

The vector

$$h(x) = \text{ReLU}(xW_1 + b_1) \in \mathbb{R}^H$$

is a new description of the input. Instead of asking the final layer to classify raw pixels, we ask it to classify learned features. Training adjusts W_1 and b_1 so that useful distinctions become easier in this hidden coordinate system (Bengio et al. 2013).

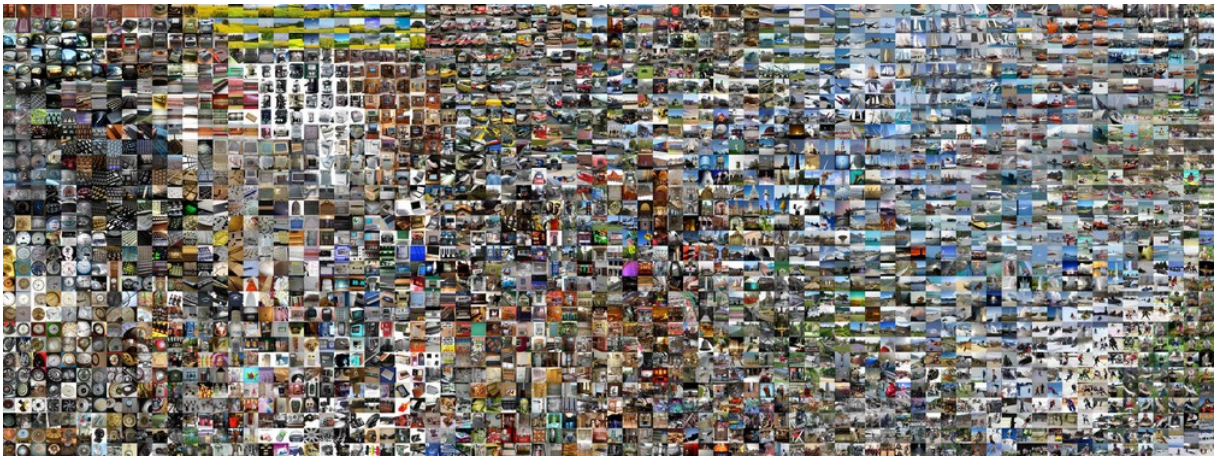
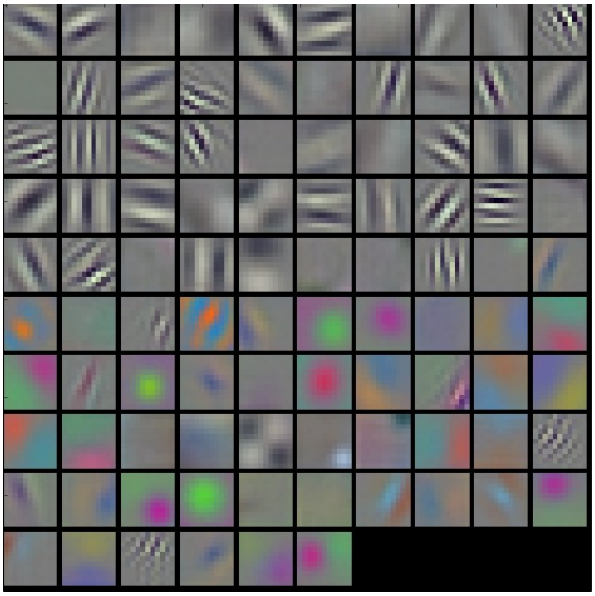
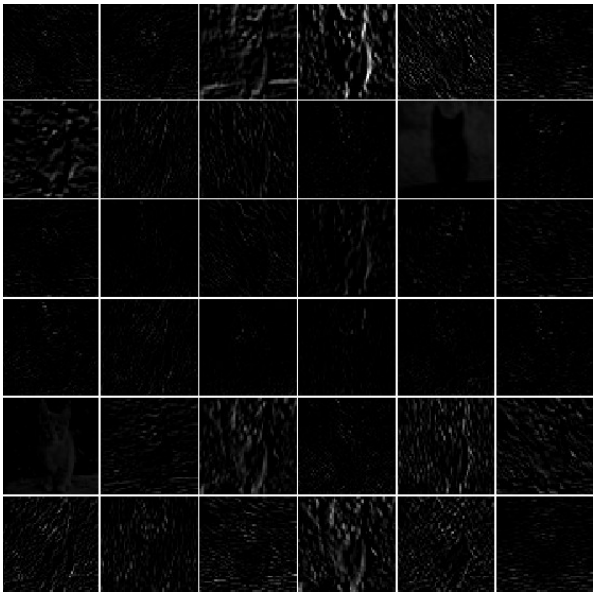


Figure 12: A t-SNE projection of AlexNet fc7 feature vectors. Each thumbnail represents one high-dimensional activation vector; images with similar learned representations appear nearby in the two-dimensional projection. Source: Stanford CS231n (Li et al., n.d.).

A two-dimensional projection is evidence, not a literal picture of the hidden space. The original vectors have thousands of components, and a projection necessarily distorts some distances. Still, the neighborhoods show that the learned coordinates capture substantial visual structure.

4.1 Filters and feature maps are different objects

A **filter** is a learned parameter. A **feature map** is the response produced when that filter is applied to a particular input. Confusing them is like confusing the weight vector w with the activation $\phi(w^\top x + b)$.

Learned AlexNet conv1 filters	Their activation maps for one cat image
	

AlexNet was a landmark large-scale convolutional classifier (Krizhevsky et al. 2012). We will study convolution later; here the visualizations make a general principle concrete: hidden units are reusable detectors, and an image is represented by the pattern of their responses.

4.2 A representation is a pattern of activations

Meaning is usually distributed across many hidden units. Two cats need not activate exactly the same units with exactly the same magnitudes, but their activation vectors may be more similar to one another than to a truck's vector.

The examples in Figure Figure 13 should not be interpreted as proof that each unit has one clean human-readable meaning. A unit may respond to several correlated visual structures, and a concept can be distributed over many units. The important object for the next layer is the complete activation vector.

4.3 Width and depth play different roles

Increasing width gives a layer more units with which to construct a piecewise-linear transformation. This can produce a more intricate boundary, but the extra capacity can fit noise as well as signal.

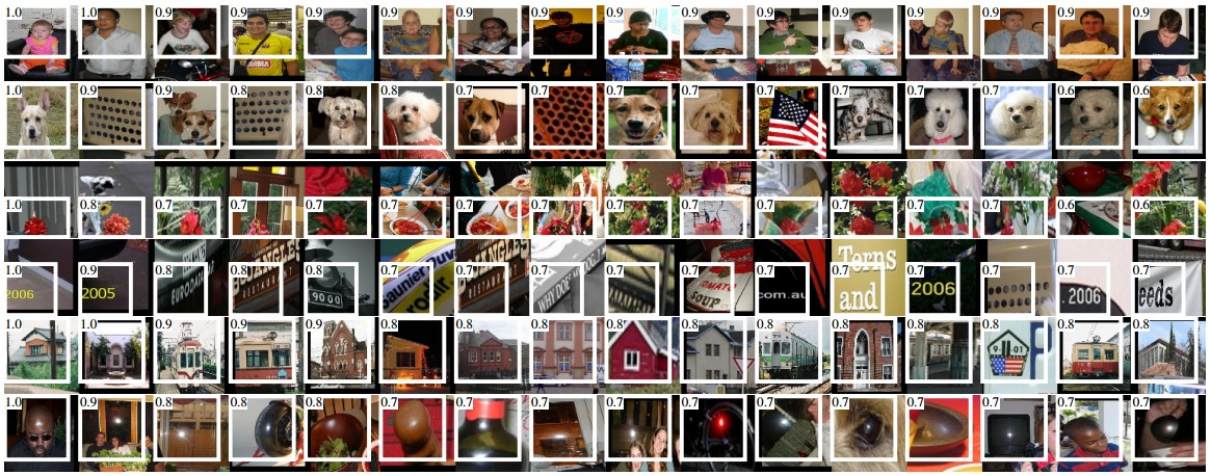


Figure 13: Image patches that maximally activate selected AlexNet pool5 units. The white values report activation magnitudes, making clear that a representation contains graded numerical responses rather than only on/off symbols. Source: Stanford CS231n (Li et al., n.d.).

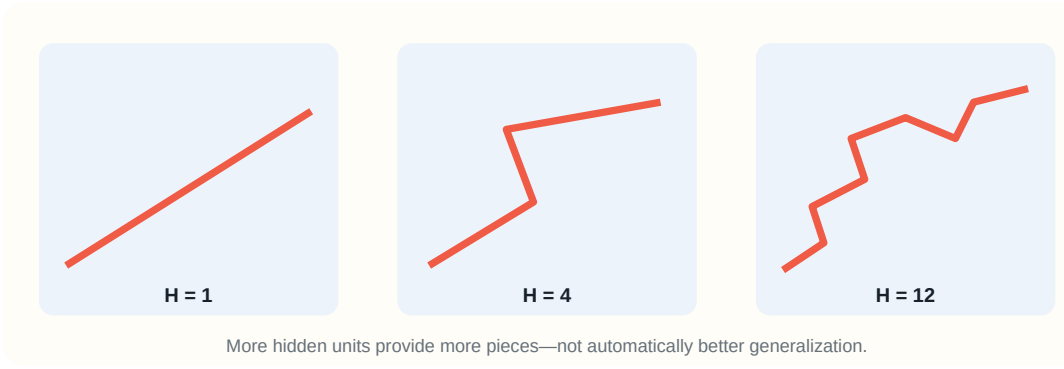
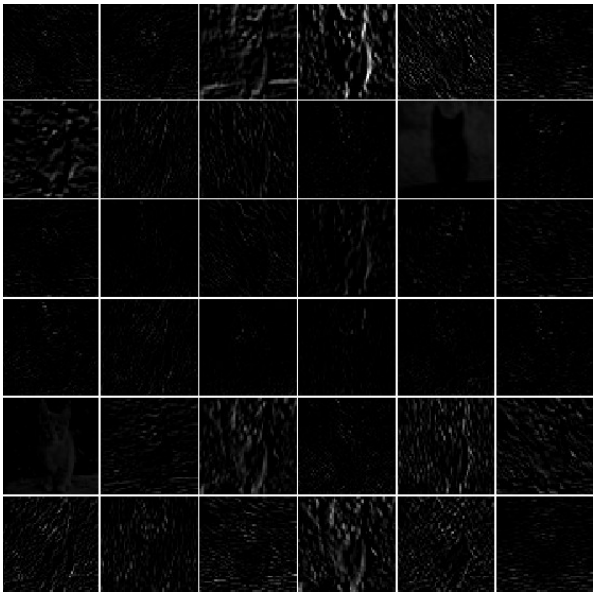
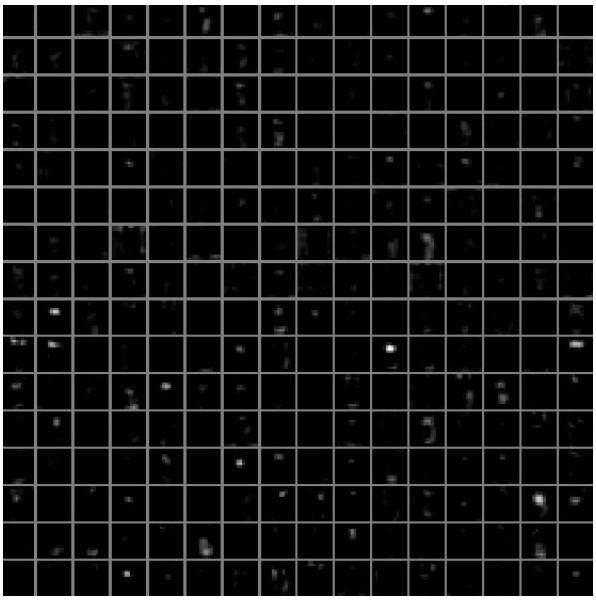


Figure 14: As hidden width grows, more rectified pieces can participate in a nonlinear boundary. Course schematic; actual behavior depends on learned parameters and optimization.

Depth repeatedly transforms one representation into another. Early learned responses often capture local contrasts and orientations; deeper responses can become more selective because they compose earlier features.

Early AlexNet layer: conv1	Deeper AlexNet layer: conv5
	

These images do not imply that every network learns the same fixed hierarchy. They illustrate what this trained model learned. Architecture, task, data, and optimization all influence the resulting representation.

5 Implementing the forward pass in NumPy

Keeping the first implementation self-contained makes the relationship between equations, arrays, and object-oriented code explicit.

```python
import numpy as np

class TwoLayerNet:
    def __init__(self, input_dim, hidden_dim, n_classes, seed=0):
        rng = np.random.default_rng(seed)
        scale = 1e-2

        self.W1 = scale * rng.standard_normal((input_dim, hidden_dim))
        self.b1 = np.zeros(hidden_dim)
        self.W2 = scale * rng.standard_normal((hidden_dim, n_classes))
        self.b2 = np.zeros(n_classes)

    def forward(self, X):
        z1 = X @ self.W1 + self.b1
```

```
        h1 = np.maximum(0.0, z1)
        scores = h1 @ self.W2 + self.b2

        cache = {"X": X, "z1": z1, "h1": h1}
        return scores, cache

    def predict(self, X):
        scores, _ = self.forward(X)
        return np.argmax(scores, axis=1)
```

The class is only an organizational container. `self.W1` means that the array belongs to this network instance and remains available across method calls. `forward` uses those stored parameters; it does not import a second hidden implementation from elsewhere.

The cache is unnecessary for prediction, but it records the intermediate values that backpropagation will need. Returning it now makes the data flow explicit.

5.1 Defensive shape checks

Silent broadcasting mistakes can produce an array with a plausible shape while implementing the wrong computation. During development, assert the contract:

```
def checked_forward(model, X):
    assert X.ndim == 2
    assert X.shape[1] == model.W1.shape[0]
    assert model.b1.shape == (model.W1.shape[1],)
    assert model.W2.shape[0] == model.W1.shape[1]

    scores, cache = model.forward(X)

    assert cache["z1"].shape == (X.shape[0], model.W1.shape[1])
    assert cache["h1"].shape == cache["z1"].shape
    assert scores.shape == (X.shape[0], model.W2.shape[1])
    return scores, cache
```

5.2 Sanity checks before training

Before trusting a loss curve, verify simple invariants:

1. `scores.shape` must be (N, K).
2. Every ReLU activation must be nonnegative.
3. `predict(X)` must return N integer class indices.
4. Identical input rows must produce identical score rows in evaluation mode.
5. With very small random weights, initial class scores should be near one another.

If the initial model assigns roughly equal probability to each of K classes, the expected softmax loss is approximately

$$-\log\left(\frac{1}{K}\right) = \log K.$$

For $K = 10$, this is about 2.303. A dramatically different initial loss is a useful signal to inspect shapes, labels, numerical stability, and initialization before writing any optimization code.

6 What the forward pass does not yet solve

The network can now produce scores, but its random hidden features are not useful yet. Learning still requires:

$$\text{scores} \longrightarrow \text{loss} \longrightarrow \text{gradients} \longrightarrow \text{parameter update}.$$

Backpropagation is the efficient procedure that computes how the loss changes with every parameter in this composed function (Rumelhart et al. 1986). The next chapters will derive that procedure and then study optimization in depth.

The forward pass is a chain of explicit transformations. Backpropagation does not replace this chain; it traverses the same computation in reverse using the chain rule.

7 Summary

- A neuron computes an affine pre-activation $z = w^\top x + b$ and a nonlinear activation $h = \phi(z)$.
- Stacking affine transformations without an activation still produces one affine transformation.
- Sigmoid and tanh saturate; ReLU is simple and non-saturating on its positive side but can become inactive.
- A two-layer classifier computes $Z_1 = XW_1 + b_1$, $H_1 = \text{ReLU}(Z_1)$, and $S = H_1W_2 + b_2$.
- Hidden activations are learned feature vectors. Their geometry can make classes easier for the final linear layer to separate.
- Filters are learned parameters; feature maps are input-dependent activations.
- Width supplies more features within a layer, while depth composes successive representations.
- Shapes and simple numerical invariants should be checked before training.

8 Reading and practice

Required · Stanford CS231n: Neural Networks Part 1

Read the notes for the class-score viewpoint, neural-network architecture, activation functions, and biological analogy (Li et al., n.d.).

Required · Deep Learning, Chapter 6

Read Deep Feedforward Networks, especially the motivation for hidden units, architecture design, and gradient-based learning (Goodfellow et al. 2016).

Visual exploration · What ConvNets learn

Explore the CS231n visualizations used in this chapter. Distinguish filters, activation maps, maximally activating inputs, and feature-space projections.

Practice · TensorFlow Playground

Open the playground. Hold all settings constant while comparing linear, sigmoid, tanh, and ReLU networks on the circle and spiral datasets.

References

Bengio, Yoshua, Aaron Courville, and Pascal Vincent. 2013. “Representation Learning: A Review and New Perspectives.” *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35 (8): 1798–828. https://doi.org/10.1109/TPAMI.2013.50.

Goodfellow, Ian, Yoshua Bengio, and Aaron Courville. 2016. *Deep Learning*. MIT Press. https://www.deeplearningbook.org/.

Krizhevsky, Alex, Ilya Sutskever, and Geoffrey E. Hinton. 2012. “ImageNet Classification with Deep Convolutional Neural Networks.” *Advances in Neural Information Processing Systems* 25. https://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks.

Li, Fei-Fei, Andrej Karpathy, and Justin Johnson. n.d. *CS231n: Deep Learning for Computer Vision—Course Notes*. https://cs231n.github.io/.

Rumelhart, David E., Geoffrey E. Hinton, and Ronald J. Williams. 1986. “Learning Representations by Back-Propagating Errors.” *Nature* 323: 533–36. https://doi.org/10.1038/323533a0.